

Aprendizado de Máquina via Álgebra Linear e Otimização

Lista de Exercícios - Módulo 3 (Árvores de Classificação Ótimas)

Prof. Alexandre Salles da Cunha e Profa. Ana Paula Couto

Junho de 2026

Data de entrega: 26 de Junho de 2026

1 Descrição da atividade

A atividade consiste em implementar o modelo para Árvores Ótimas discutido em sala de aula, descrito em [1] e reapresentado neste documento. Como sugestão, recomendamos o uso de uma linguagem algébrica como `pyomo` para modelagem do problema e, para resolver o problema após instanciado, os pacotes `CPLEX`, `GUROBI`, ou `XPRESS`. Caso o aluno não disponha de algum destes pacotes, ainda que na versão acadêmica para estudantes ou no laboratório do DCC ao qual faça parte (caso faça parte de algum), sugerimos o uso do `GLPK`. Este último é disponível para download, mas o seu desempenho numérico é ordens de magnitude inferior ao dos pacotes comerciais que foram citados acima.

O objetivo principal da atividade é comparar, em dados sintéticos, uma abordagem global de otimização para a construção de árvores de classificação com a heurística gulosa CART. A atividade também deve permitir que os alunos observem o efeito da profundidade da árvore, do tamanho da amostra, do número de atributos e do parâmetro de penalização por complexidade sobre a qualidade das soluções e sobre o esforço computacional requerido pelo modelo de programação inteira.

2 Revisão do modelo a ser implementado

O modelo de Árvores Ótimas de Classificação é reapresentado abaixo. Este é o modelo que deve ser implementado e testado. No modelo, T_B denota o conjunto de nós internos, ou nós de ramificação, e T_L denota o conjunto de folhas. Para cada folha t , os conjuntos $A_L(t)$ e $A_R(t)$ indicam, respectivamente, os ancestrais de t nos quais o caminho até t segue pela ramificação esquerda ou pela ramificação direita. O parâmetro D denota a profundidade máxima da árvore usada pelo modelo de otimização.

$$\min \quad \frac{1}{\hat{L}} \sum_{t \in T_L} L_t + \alpha \sum_{m \in T_B} d_m \quad (1)$$

s.t. :

$$d_m = \sum_{j \in [p]} a_{jm} \quad m \in T_B \quad (2)$$

$$b_m \leq d_m \quad m \in T_B \quad (3)$$

$$0 \leq b_m \quad m \in T_B \quad (4)$$

$$d_m \leq d_{p(m)} \quad m \in T_B \setminus \{1\} \quad (5)$$

$$1 = \sum_{t \in T_L} z_{it} \quad i \in [n] \quad (6)$$

$$z_{it} \leq l_t \quad i \in [n], t \in T_L \quad (7)$$

$$N_{min} l_t \leq \sum_{i \in [n]} z_{it} \quad t \in T_L \quad (8)$$

$$a_m^T x_i \geq b_m - (1 - z_{it}) \quad m \in A_R(t), i \in [n], t \in T_L \quad (9)$$

$$a_m^T (x_i + \epsilon) \leq b_m + (1 + \epsilon_{max})(1 - z_{it}) \quad m \in A_L(t), i \in [n], t \in T_L \quad (10)$$

$$d_m \geq \sum_{t \in T_L^{eq}(m)} z_{it} \quad m \in T_B, i \in [n] \quad (11)$$

$$N_{kt} = \sum_{i: y_i = k} z_{it} \quad k \in [K], t \in T_L \quad (12)$$

$$N_t = \sum_{i \in [n]} z_{it} \quad t \in T_L \quad (13)$$

$$l_t = \sum_{k \in [K]} c_{kt} \quad t \in T_L \quad (14)$$

$$L_t \geq N_t - N_{kt} - n(1 - c_{kt}) \quad t \in T_L, k \in [K] \quad (15)$$

$$L_t \leq N_t - N_{kt} + n c_{kt} \quad t \in T_L, k \in [K] \quad (16)$$

$$L_t \geq 0 \quad t \in T_L \quad (17)$$

$$c_{kt} \in \{0, 1\} \quad k \in [K], t \in T_L \quad (18)$$

$$N_{kt} \geq 0 \quad k \in [K], t \in T_L \quad (19)$$

$$N_t \geq 0 \quad t \in T_L \quad (20)$$

$$l_t \in \{0, 1\} \quad t \in T_L \quad (21)$$

$$d_m \in \{0, 1\} \quad m \in T_B \quad (22)$$

$$z_{it} \in \{0, 1\} \quad i \in [n], t \in T_L \quad (23)$$

$$a_{jm} \in \{0, 1\} \quad j \in [p], m \in T_B. \quad (24)$$

Neste trabalho, os atributos devem ser normalizados para o intervalo $[0, 1]$. Nos dados sintéticos gerados conforme descrito a seguir, esta normalização já é satisfeita por construção.

3 Descrição dos testes computacionais

Os testes computacionais devem ser realizados com datasets sintéticos inspirados nos experimentos descritos no artigo, cuja construção é explicada abaixo.

3.1 Aspectos gerais

Os dados de treinamento serão gerados a partir de uma árvore de classificação construída de altura D_{true} estabelecida (sem nenhum dado de treinamento prévio) que representará a *verdade absoluta*. Esta árvore é a árvore que deve ser aprendida.

Dados D_{true} e p , o número de atributos, a árvore de altura D_{true} é construída distribuindo-se splits $a_{jm} = 1$ para exatamente um $j \in \{1, \dots, p\}$ e todo $m \in T_B$, e escolhendo-se $b_m \in [0, 1]$, de forma aleatória, em cada um dos nós internos (nós branching).

A cada uma das folhas $t \in T_L$ da árvore é atribuída uma classe do conjunto $\{1, \dots, K\}$, de forma que duas folhas que compartilhem o mesmo pai não recebam a mesma classe, caso contrário este nó pai poderia ser substituído por uma folha.

Uma vez construída a árvore (alocação de splits e alocação de classes às folhas) são gerados indivíduos $x_i \in \mathbb{R}^p : i = 1, \dots, n$ aleatoriamente, sorteando-se valores no intervalo $[0, 1]$ para cada um dos p atributos de cada indivíduo. Na sequência, cada indivíduo é classificado percorrendo o caminho da raiz da árvore até alguma folha. A classe da folha à qual o indivíduo x_i foi roteado é o valor atribuído ao dado y_i .

O tamanho total dos dados gerados deve ser $n \in \{100, 200\}$. Cerca de 80% dos dados deve ser reservado para treinamento e o restante para validação. O conjunto de validação não deve ser usado para escolher α , D , N_{min} ou qualquer outro parâmetro. Ele deve ser usado apenas para medir o desempenho fora da amostra das árvores obtidas.

Para cada experimento, $ntrees = 5$ árvores de classificação de *verdade absoluta* e os correspondentes dados (x_i, y_i) devem ser gerados. Deve-se assumir $K = 2$, $p \in \{2, 6\}$ e $D_{true} = 3$. Para este experimento específico, usamos $N_{min} = 1$ e dois valores de $\alpha = 0$ e $\alpha = \frac{2}{n}$. Os valores de D usados para o modelo devem variar de $D = 1, \dots, D_{true}$.

Quando $D = D_{true}$, existe uma solução factível com erro de classificação zero para os dados de treinamento, pois estes dados foram gerados por uma árvore de profundidade D_{true} , sem ruído. Caso o resolvidor não encontre uma solução com erro zero dentro do limite de tempo estabelecido, o aluno deve discutir se isso se deve a uma dificuldade computacional, a uma limitação do resolvidor ou a algum detalhe de implementação. Quando $D < D_{true}$, o modelo não tem necessariamente capacidade para representar exatamente a árvore verdadeira. Neste caso, os erros da árvore obtida com o modelo devem ser medidos tanto nos dados de treinamento quanto nos dados de validação.

Para cada um dos algoritmos (árvore ótima via modelo e CART), as seguintes estatísticas devem ser geradas:

1. Quando $D < D_{true}$, devemos comparar os erros de treinamento e de validação da árvore obtida com o modelo e com a heurística CART, verificando o efeito do aumento de n e de p .

2. Quando $D = D_{true}$, devemos verificar se o modelo encontrou uma solução ótima alternativa ou se recuperou exatamente a árvore verdadeira. Como pode haver mais de uma árvore que classifique corretamente todos os pontos amostrados, a recuperação exata da estrutura verdadeira não deve ser confundida com erro de treinamento zero.

Para cada instanciã o (escolha de D, p, n, α), o algoritmo do modelo de otimiza  o deve rodar por n o mais de 20 minutos. Ap s este tempo, a melhor solu  o encontrada deve ser recuperada (ainda que n o seja a  tima). O percentual de inst ncias que foram resolvidas   otimalidade deve ser informado para cada instanci  o. Alunos ou grupos com acesso a resolvers comerciais e maior capacidade computacional podem, opcionalmente, aumentar o limite de tempo para at  1 hora em alguns experimentos adicionais.

Os resultados obtidos com o algoritmo exato devem ser comparados com os resultados da heur stica CART (por exemplo dispon vel na biblioteca python `from sklearn.tree import DecisionTreeClassifier`). Para tornar a compara  o correta e a solu  o da heur stica compat vel com o modelo, deve-se ajustar o par metro N_{min} e a profundidade. Por exemplo, para $N_{min} = 1$ e $D = 3$ ter amos:

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,          # Profundidade maxima igual a 3
    min_samples_leaf=1,   # Mesmo valor de N_min usado no modelo
    random_state=42
)
```

4 Formato de apresenta  o do trabalho

O aluno deve apresentar um relat rio contendo as m dias dos erros de treinamento e valida  o, para cada instanci  o D, p, n, α e cada um dos dois algoritmos. Deve tamb m analisar o comportamento dos dois algoritmos em fun  o da varia  o da instanci  o. Por exemplo: qual o impacto da varia  o de α, n, D e p no desempenho relativo dos dois algoritmos? Tamb m devem ser informadas as estat sticas do resolver de programa  o inteira empregado para resolver o modelo. Devem ser informados, para cada D, p, n, α , quantos casos foram resolvidos   otimalidade, os tempos m dios e m ximos de computa  o e o gap m dio de otimalidade ao final do algoritmo, para as inst ncias n o resolvidas   otimalidade.

O relat rio tamb m deve conter uma breve discuss o qualitativa das  rvores obtidas. Em particular, deve-se comentar se o aumento de D reduz o erro de treinamento, se esta redu  o tamb m aparece no conjunto de valida  o, se a penaliza  o por complexidade altera o n mero de ramifica  es usadas, e em quais situa  es o modelo de otimiza  o apresenta vantagem sobre a heur stica CART.

Referências

- [1] Dimitris Bertsimas and Jack Dunn. Optimal classification trees. *Machine Learning*, 106:1039–1082, 2017.